
INITIATION AU MACHINE LEARNING

RAPPORT DE SYNTHÈSE

Parcours Kaggle

Octobre 2025

CLAVERIE TIMM

Établissement : SEATECH

Matière : ATELIER IA



Table des matières

1	Introduction au Machine Learning	3
1.1	Chargement et Préparation des Données	3
1.2	Validation du Modèle	3
1.3	Modélisation : Arbre de Décision	4
1.4	Optimisation : Ajustement des Hyperparamètres	4
1.5	Modélisation Avancée : Forêt Random Forest	5
2	Concepts Intermédiaires en Machine Learning	6
2.1	Gestion des Données Manquantes	6
2.1.1	Suppression des Colonnes	6
2.1.2	Imputation Simple	7
2.1.3	Imputation avec Extension	7
2.2	Gestion des Variables Catégorielles	8
2.2.1	Suppression des Variables Catégorielles	8
2.2.2	Encodage Ordinal	8
2.2.3	Encodage "One-Hot"	9
2.3	Pipelines	9
2.3.1	Définir les Étapes de Prétraitement	10
2.3.2	Définir le Modèle	10
2.3.3	Créer et Évaluer le Pipeline	10
2.4	Validation Croisée (Cross-Validation)	11
2.5	Gradient Boosting (XGBoost)	11
2.6	Fuite de Données (Data Leakage)	12
2.6.1	Fuite de Cible (Target Leakage)	12
2.6.2	Contamination Train-Test	13
3	Explicabilité des Modèles (Machine Learning Explainability)	14
3.1	Introduction	14
3.2	"Importance par Permutation"	14
3.3	Graphiques de Dépendance Partielle (Partial Plots)	15
3.3.1	PDP à une dimension (1D)	15
3.3.2	PDP à deux dimensions (2D)	15
3.3.3	Interprétation	16
3.4	Valeurs SHAP (SHAP Values)	16
3.4.1	SHAP Force Plot	16
3.4.2	Interprétation	17
3.5	Usages Avancés des Valeurs SHAP	17
3.5.1	SHAP Summary Plot	17
3.5.2	SHAP Dependence Plot	18

NOTES :

Ce rapport rassemble mes notes de cours, des exemples et ce que j'ai appris de plusieurs formations en ligne.

Mes notes m'ont permis de travailler sur plusieurs exercices, mini projets et projet personnel, mais pas de résumer toutes les connaissances apprises de manière structurée.

Le rapport qui suit a été rédigé par IA, il reprend EXCLUSIVEMENT mes notes. Je précise, juste la mise en page et la structuration du document m'est propre. Le rapport a été relu et corrigé par mes soins.

Ce document résume ainsi de manière succincte un ensemble de connaissances et est conçu pour servir de base solide à un projet de Machine Learning.

Je suis entièrement conscient que le Machine Learning ne représente qu'une petite partie de ce que l'on peut faire avec l'Intelligence Artificielle. L'objectif de ces travaux personnels est de me faire découvrir plusieurs méthodes de résolution de problèmes.

1 Introduction au Machine Learning

Ce chapitre synthétise le flux de travail (workflow) de base en Machine Learning, basé sur les cours de la plateforme Kaggle. Nous aborderons le chargement des données, la création d'un modèle, son évaluation et son optimisation.

1.1 Chargement et Préparation des Données

La première étape consiste à charger les données. Nous utilisons la bibliothèque `pandas` pour importer les fichiers CSV.

```
1 import pandas as pd
2
3 # Chemin vers le fichier de donnees (exemple Iowa)
4 iowa_file_path = '../input/home-data-for-ml-course/train.csv'
5 # Lecture du fichier CSV dans un DataFrame pandas
6 home_data = pd.read_csv(iowa_file_path)
```

Listing 1.1 – "Importation de pandas et chargement des donnees"

Une fois les données chargées, nous devons définir notre objectif (la cible) et les informations que nous utiliserons pour la prédiction (les caractéristiques).

Définition de la Cible (y) La cible est la variable que nous essayons de prédire. Dans cet exemple, il s'agit du prix de vente (`SalePrice`).

```
1 y = home_data.SalePrice
```

Listing 1.2 – "Selection de la cible de prediction"

Sélection des Caractéristiques (X) Les caractéristiques (features) sont les colonnes utilisées pour construire notre modèle. Nous les sélectionnons dans une liste.

```
1 # Creation de la liste des noms de caracteristiques
2 feature_names = ['LotArea', 'YearBuilt', '1stFlrSF', '2ndFlrSF', 'FullBath', 'BedroomAbvGr', '
    TotRmsAbvGrd']
3
4 # Selection des donnees correspondantes
5 X = home_data[feature_names]
```

Listing 1.3 – "Selection des caracteristiques"

On peut ensuite inspecter nos données, par exemple avec `X.describe()` ou `X.head()`.

1.2 Validation du Modèle

Pour évaluer la performance réelle d'un modèle, il ne faut jamais l'évaluer sur les données qu'il a utilisées pour s'entraîner. Nous divisons donc notre jeu de données en deux : un ensemble d'entraînement (train) et un ensemble de validation (val).

Nous utilisons `train_test_split` de `scikit-learn`. L'argument `random_state` garantit que nous obtenons la même division à chaque exécution.

```
1 from sklearn.model_selection import train_test_split
2
3 # Division de X et y
4 # Nous utilisons random_state=1 pour la reproductibilite
5 train_X, val_X, train_y, val_y = train_test_split(X, y, random_state = 1)
```

Listing 1.4 – "Division des donnees en ensembles d'entrainement et de validation"

1.3 Modélisation : Arbre de Décision

Nous commençons par un modèle simple : l'Arbre de Décision (Decision Tree Regressor).

Entraînement du modèle Nous créons une instance du modèle et l'entraînons sur les données d'entraînement (`train_X`, `train_y`) en utilisant la méthode `.fit()`.

```
1 from sklearn.tree import DecisionTreeRegressor
2
3 # Définir le modele. Specifier random_state pour la reproductibilite
4 iowa_model = DecisionTreeRegressor(random_state=1)
5
6 # Entraîner le modele
7 iowa_model.fit(train_X, train_y)
```

Listing 1.5 – "Definition et entrainement de l'Arbre de Decision"

Évaluation du modèle Nous évaluons le modèle en faisant des prédictions sur l'ensemble de validation (`val_X`) et en comparant ces prédictions (`preds_val`) aux vraies valeurs (`val_y`). Nous utilisons l'Erreur Absolue Moyenne (MAE).

```
1 from sklearn.metrics import mean_absolute_error
2
3 # Faire des predictions sur les donnees de validation
4 val_predictions = iowa_model.predict(val_X)
5
6 # Calculer le MAE
7 mae = mean_absolute_error(val_y, val_predictions)
8 print("Validation MAE (Arbre de Decision simple): {}".format(mae))
```

Listing 1.6 – "Evaluation du modele avec le MAE"

1.4 Optimisation : Ajustement des Hyperparamètres

La performance d'un arbre de décision dépend de sa profondeur. Un arbre trop profond (trop de feuilles) "sur-apprend" (overfitting) les données d'entraînement. Nous pouvons contrôler cela avec l'hyperparamètre `max_leaf_nodes`.

Nous créons une fonction pour calculer le MAE pour un nombre de feuilles donné.

```
1 def get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y):
2     model = DecisionTreeRegressor(max_leaf_nodes=max_leaf_nodes, random_state=0)
3     model.fit(train_X, train_y)
4     preds_val = model.predict(val_X)
5     mae = mean_absolute_error(val_y, preds_val)
6     return(mae)
```

Listing 1.7 – "Fonction pour calculer le MAE en fonction de max_leaf_nodes"

Nous testons ensuite plusieurs valeurs pour trouver la meilleure.

```
1 # Boucle sur différentes valeurs de max_leaf_nodes
2 for max_leaf_nodes in [5, 50, 500, 5000]:
3     my_mae = get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y)
4     print("Max leaf nodes: %d \t\t Mean Absolute Error:  %d" %(max_leaf_nodes, my_mae))
```

Listing 1.8 – "Comparaison des MAE pour différentes tailles d'arbre"

1.5 Modélisation Avancée : Forêt Random Forest

La Forêt Aléatoire (Random Forest) est un modèle plus avancé qui utilise plusieurs arbres de décision pour améliorer la précision et réduire l'overfitting.

Nous l'entraînons de la même manière que l'arbre de décision, en utilisant `RandomForestRegressor`.

```
1 from sklearn.ensemble import RandomForestRegressor
2
3 # Definition du modele
4 rf_model = RandomForestRegressor(random_state=1)
5
6 # Entrainement du modele
7 rf_model.fit(train_X, train_y)
8
9 # Evaluation du MAE sur les donnees de validation
10 rf_val_preds = rf_model.predict(val_X)
11 rf_val_mae = mean_absolute_error(val_y, rf_val_preds)
12
13 print("Validation MAE for Random Forest Model: {}".format(rf_val_mae))
```

Listing 1.9 – "Entraînement et évaluation d'un modèle Random Forest"

En général, le modèle Random Forest devrait produire un MAE plus faible que l'arbre de décision optimisé, démontrant ainsi sa supériorité pour ce type de problème.

2 Concepts Intermédiaires en Machine Learning

Ce chapitre explore les techniques essentielles pour améliorer la performance des modèles. Nous aborderons le traitement des données manquantes, l'encodage des variables catégorielles, l'utilisation de pipelines pour industrialiser le processus, la validation croisée, et les modèles avancés de type Gradient Boosting. Enfin, nous analyserons le piège le plus critique en ML : la fuite de données.

2.1 Gestion des Données Manquantes

Les jeux de données réels contiennent presque toujours des valeurs manquantes. Une investigation préliminaire est nécessaire pour identifier les colonnes concernées et le volume de données manquantes.

```
1 # Forme des donnees (lignes, colonnes)
2 print(X_train.shape)
3
4 # Nombre de valeurs manquantes par colonne
5 missing_val_count_by_column = (X_train.isnull().sum())
6 print(missing_val_count_by_column[missing_val_count_by_column > 0])
```

Listing 2.1 – "Investigation préliminaire des données manquantes"

Il existe trois approches principales pour les gérer, que nous pouvons comparer grâce à une fonction évaluant le MAE (Mean Absolute Error).

2.1.1 Suppression des Colonnes

La méthode la plus simple consiste à supprimer entièrement les colonnes qui contiennent des valeurs manquantes.

```
1 # Obtenir les noms des colonnes avec des valeurs manquantes
2 cols_with_missing = [col for col in X_train.columns
3                       if X_train[col].isnull().any()]
4
5 # Supprimer ces colonnes des ensembles d'entraînement et de validation
6 reduced_X_train = X_train.drop(cols_with_missing, axis=1)
7 reduced_X_valid = X_valid.drop(cols_with_missing, axis=1)
8
9 # print(score_dataset(reduced_X_train, reduced_X_valid, y_train, y_valid))
```

Listing 2.2 – "Approche 1 : Suppression des colonnes avec valeurs manquantes"

Méthodologie Cette approche est radicale et n'est recommandée que si les colonnes en question contiennent peu d'information pertinente ou si la majorité de leurs données est manquante. Elle entraîne souvent une perte de performance.

2.1.2 Imputation Simple

L'imputation consiste à remplacer les valeurs manquantes par une valeur statistique (moyenne, médiane, mode). Nous utilisons `SimpleImputer` de `scikit-learn`.

```
1 from sklearn.impute import SimpleImputer
2
3 # Imputation (par défaut : moyenne, 'median' est aussi une option)
4 my_imputer = SimpleImputer(strategy='median')
5 imputed_X_train = pd.DataFrame(my_imputer.fit_transform(X_train))
6 imputed_X_valid = pd.DataFrame(my_imputer.transform(X_valid))
7
8 # L'imputation supprime les noms de colonnes, il faut les remettre
9 imputed_X_train.columns = X_train.columns
10 imputed_X_valid.columns = X_valid.columns
11
12 # print(score_dataset(imputed_X_train, imputed_X_valid, y_train, y_valid))
```

Listing 2.3 – "Approche 2 : Imputation (par moyenne ou mediane)"

Méthodologie L'imputation conserve toutes les colonnes et est généralement plus performante que la suppression. Le choix de la stratégie (moyenne, médiane) dépend de la distribution des données.

2.1.3 Imputation avec Extension

Cette approche améliore l'imputation simple. Elle ajoute une nouvelle colonne booléenne pour indiquer au modèle quelles valeurs étaient *originellement* manquantes. Cela permet au modèle d'apprendre si le fait qu'une valeur soit manquante est en soi une information prédictive.

```
1 # Copie des donnees
2 X_train_plus = X_train.copy()
3 X_valid_plus = X_valid.copy()
4
5 # Ajout des nouvelles colonnes "was_missing"
6 for col in cols_with_missing:
7     X_train_plus[col + '_was_missing'] = X_train_plus[col].isnull()
8     X_valid_plus[col + '_was_missing'] = X_valid_plus[col].isnull()
9
10 # Imputation
11 my_imputer = SimpleImputer()
12 imputed_X_train_plus = pd.DataFrame(my_imputer.fit_transform(X_train_plus))
13 imputed_X_valid_plus = pd.DataFrame(my_imputer.transform(X_valid_plus))
14
15 # Restaurer les noms de colonnes
16 imputed_X_train_plus.columns = X_train_plus.columns
17 imputed_X_valid_plus.columns = X_valid_plus.columns
18
19 # print(score_dataset(imputed_X_train_plus, imputed_X_valid_plus, y_train, y_valid))
```

Listing 2.4 – "Approche 3 : Imputation et ajout de colonnes indicatrices"

Méthodologie Cette approche est souvent la plus performante car elle combine la robustesse de l'imputation avec une information contextuelle précieuse pour le modèle.

2.2 Gestion des Variables Catégorielles

Les modèles de machine learning nécessitent des entrées numériques. Les variables catégorielles (de type object) doivent être encodées.

Cardinalité La "cardinalité" désigne le nombre de valeurs uniques dans une colonne. Il est courant de traiter différemment les colonnes à faible cardinalité (ex : < 10 valeurs uniques) et celles à haute cardinalité.

```
1 # Sélectionner colonnes catégorielles a faible cardinalite
2 low_cardinality_cols = [cname for cname in X_train_full.columns
3                          if X_train_full[cname].nunique() < 10 and
4                          X_train_full[cname].dtype == "object"]
5
6 # Sélectionner colonnes numériques
7 numerical_cols = [cname for cname in X_train_full.columns
8                  if X_train_full[cname].dtype in ['int64', 'float64']]
9
10 # Garder uniquement les colonnes selectionnees
11 my_cols = low_cardinality_cols + numerical_cols
12 X_train = X_train_full[my_cols].copy()
13 X_valid = X_valid_full[my_cols].copy()
```

Listing 2.5 – "Selection des colonnes catégorielles et numériques"

2.2.1 Suppression des Variables Catégorielles

La méthode la plus simple est de les ignorer et de ne garder que les colonnes numériques.

```
1 drop_X_train = X_train.select_dtypes(exclude=['object'])
2 drop_X_valid = X_valid.select_dtypes(exclude=['object'])
3 % print(score_dataset(drop_X_train, drop_X_valid, y_train, y_valid))
```

Listing 2.6 – "Approche 1 : Suppression des variables catégorielles"

2.2.2 Encodage Ordinal

L'encodage ordinal assigne un entier unique à chaque valeur (ex : "Mauvais" -> 0, "Bon" -> 1, "Excellent" -> 2). Il est adapté pour les variables qui ont un ordre logique.

```
1 from sklearn.preprocessing import OrdinalEncoder
2
3 # Copie pour eviter de modifier les donnees originales
4 label_X_train = X_train.copy()
5 label_X_valid = X_valid.copy()
6
7 # Appliquer l'encodeur aux colonnes catégorielles (object_cols)
8 ordinal_encoder = OrdinalEncoder()
9 label_X_train[object_cols] = ordinal_encoder.fit_transform(X_train[object_cols])
10 label_X_valid[object_cols] = ordinal_encoder.transform(X_valid[object_cols])
11 % print(score_dataset(label_X_train, label_X_valid, y_train, y_valid))
```

Listing 2.7 – "Approche 2 : Encodage Ordinal"

Méthodologie Le risque de cette approche est que si l'ensemble de validation contient des valeurs qui n'étaient pas présentes dans l'ensemble d'entraînement, l'encodeur générera une erreur. Il faut donc s'assurer que les colonnes encodées ordinalement ont des ensembles de valeurs cohérents entre le train et le valid set.

2.2.3 Encodage "One-Hot"

L'encodage "One-Hot" crée de nouvelles colonnes binaires pour chaque valeur unique. C'est l'approche la plus courante pour les variables nominales (sans ordre).

```
1 from sklearn.preprocessing import OneHotEncoder
2
3 # Appliquer l'encodeur One-Hot
4 # handle_unknown='ignore' évite les erreurs sur valeurs inconnues
5 OH_encoder = OneHotEncoder(handle_unknown='ignore', sparse=False)
6 OH_cols_train = pd.DataFrame(OH_encoder.fit_transform(X_train[object_cols]))
7 OH_cols_valid = pd.DataFrame(OH_encoder.transform(X_valid[object_cols]))
8
9 # L'encodage One-Hot supprime l'index, il faut le remettre
10 OH_cols_train.index = X_train.index
11 OH_cols_valid.index = X_valid.index
12
13 # Supprimer les colonnes catégorielles originales
14 num_X_train = X_train.drop(object_cols, axis=1)
15 num_X_valid = X_valid.drop(object_cols, axis=1)
16
17 # Ajouter les nouvelles colonnes encodées aux données numériques
18 OH_X_train = pd.concat([num_X_train, OH_cols_train], axis=1)
19 OH_X_valid = pd.concat([num_X_valid, OH_cols_valid], axis=1)
20
21 # S'assurer que les noms de colonnes sont des strings
22 OH_X_train.columns = OH_X_train.columns.astype(str)
23 OH_X_valid.columns = OH_X_valid.columns.astype(str)
```

Listing 2.8 – "Approche 3 : Encodage One-Hot"

2.3 Pipelines

Les pipelines sont un outil puissant de `scikit-learn` pour organiser et chaîner les étapes de prétraitement et de modélisation.

Avantages des Pipelines :

- **Code plus propre** : Évite d'avoir à gérer manuellement les `X_train` et `X_valid` à chaque étape de prétraitement.
- **Moins de bugs** : Réduit le risque d'oublier une étape ou de l'appliquer incorrectement (par exemple, "fitter" un imputer sur les données de validation).
- **Prêt pour la production** : Facilite grandement le déploiement du modèle.
- **Anti-contamination** : Empêche la "contamination train-test" (voir section 2.6).

2.3.1 Définir les Étapes de Prétraitement

Nous utilisons un `ColumnTransformer` pour appliquer différents traitements aux colonnes numériques et catégorielles.

```
1 from sklearn.compose import ColumnTransformer
2 from sklearn.pipeline import Pipeline
3 from sklearn.impute import SimpleImputer
4 from sklearn.preprocessing import OneHotEncoder
5
6 # Prétraitement pour les données numériques
7 # (Imputation simple avec une constante)
8 numerical_transformer = SimpleImputer(strategy='constant')
9
10 # Prétraitement pour les données catégorielles
11 # (Imputation + One-Hot)
12 categorical_transformer = Pipeline(steps=[
13     ('imputer', SimpleImputer(strategy='most_frequent')),
14     ('onehot', OneHotEncoder(handle_unknown='ignore'))
15 ])
16
17 # Assemblage des pre-traitements
18 preprocessor = ColumnTransformer(
19     transformers=[
20         ('num', numerical_transformer, numerical_cols),
21         ('cat', categorical_transformer, categorical_cols)
22     ])
```

Listing 2.9 – "Etape 1 : Definition du pre-processeur"

2.3.2 Définir le Modèle

On choisit le modèle qui sera la dernière étape du pipeline.

```
1 from sklearn.ensemble import RandomForestRegressor
2 model = RandomForestRegressor(n_estimators=100, random_state=0)
```

Listing 2.10 – "Etape 2 : Definition du modele"

2.3.3 Créer et Évaluer le Pipeline

On assemble le pré-processeur et le modèle dans un `Pipeline`. Désormais, les appels `.fit()` et `.predict()` gèrent l'ensemble du processus automatiquement.

```
1 from sklearn.metrics import mean_absolute_error
2
3 # Assemblage du pipeline
4 my_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
5                               ('model', model)
6                               ])
7
8 # Entraînement (applique le pre-traitement à X_train puis entraîne le modele)
9 my_pipeline.fit(X_train, y_train)
10
11 # Prediction (applique le pre-traitement à X_valid puis predit)
12 preds = my_pipeline.predict(X_valid)
```

```
13
14 # Evaluation
15 score = mean_absolute_error(y_valid, preds)
16 print('MAE:', score)
```

Listing 2.11 – "Etape 3 : Creation et evaluation du pipeline"

2.4 Validation Croisée (Cross-Validation)

La validation croisée (CV) offre une mesure plus robuste de la performance du modèle que la simple division `train_test_split`. Elle divise les données en N "plis" (folds), entraîne le modèle N fois en utilisant à chaque fois un pli différent pour la validation, et calcule la moyenne des scores.

Méthodologie La CV est particulièrement utile sur les **petits jeux de données**. Pour les grands jeux de données, une simple division de validation suffit et est plus rapide.

```
1 from sklearn.model_selection import cross_val_score
2
3 # Definition d'un pipeline
4 my_pipeline = Pipeline(steps=[('preprocessor', SimpleImputer()),
5                               ('model', RandomForestRegressor(n_estimators=50,
6                                                             random_state=0))
7                               ])
8
9 # cross_val_score calcule un score negatif, on multiplie par -1
10 scores = -1 * cross_val_score(my_pipeline, X, y,
11                               cv=5, % 5 plis
12                               scoring='neg_mean_absolute_error')
13
14 print("Scores MAE:", scores)
15 print("Moyenne MAE:", scores.mean())
```

Listing 2.12 – "Utilisation de `cross_val_score` avec un pipeline"

La CV est excellente pour tester l'impact d'un hyperparamètre. Par exemple, tester le nombre d'estimateurs (`n_estimators`).

2.5 Gradient Boosting (XGBoost)

Le Gradient Boosting (Extreme Gradient Boosting - XGBoost) est une méthode d'ensemble, comme le Random Forest, mais qui fonctionne différemment. Au lieu de moyenner des arbres indépendants, il construit les arbres de manière **itérative** : chaque nouvel arbre est entraîné pour corriger les erreurs de l'arbre précédent.

Méthodologie et Hyperparamètres Clés

- `n_estimators` : Le nombre d'arbres (cycles) à construire. Trop peu mène au sous-ajustement (underfitting), trop mène au sur-ajustement (overfitting).
- `early_stopping_rounds` : Une méthode pratique pour trouver le `n_estimators` optimal. Le modèle arrête l'entraînement si le score sur les données de validation (`eval_set`) ne s'améliore pas pendant N cycles consécutifs.

- `learning_rate` : Un faible taux d'apprentissage (ex : 0.05) "ralentit" l'apprentissage, obligeant le modèle à être plus prudent. Cela nécessite généralement plus de `n_estimators` mais conduit souvent à de meilleurs résultats.
- `n_jobs` : Permet de paralléliser l'entraînement (à régler sur le nombre de cœurs de votre machine). Utile sur de grands jeux de données.

```
1 from xgboost import XGBRegressor
2
3 # Definition du modele avec un n_estimators eleve
4 my_model = XGBRegressor(n_estimators=1000, learning_rate=0.05, n_jobs=4)
5
6 # Entrainement avec early stopping
7 my_model.fit(X_train, y_train,
8             early_stopping_rounds=5, % Arrete apres 5 tours sans amelioration
9             eval_set=[(X_valid, y_valid)], % Donnees pour l'evaluation
10             verbose=False) % N'imprime pas les etapes
11
12 % predictions = my_model.predict(X_valid)
13 % print("MAE: " + str(mean_absolute_error(predictions, y_valid)))
```

Listing 2.13 – "Exemple de modele XGBoost avec early stopping"

2.6 Fuite de Données (Data Leakage)

La fuite de données est le piège le plus dangereux en ML. Elle se produit lorsque le modèle a accès, pendant l'entraînement, à des informations qui ne seront pas disponibles au moment de la prédiction. Le modèle semble alors extrêmement précis, mais échoue totalement en production.

2.6.1 Fuite de Cible (Target Leakage)

La fuite de cible se produit lorsque vos caractéristiques incluent des données "du futur" ou des conséquences directes de la cible que vous essayez de prédire.

Méthodologie : Penser en termes de chronologie La clé est de réfléchir à l'ordre chronologique dans lequel les données deviennent disponibles.

Exemple 1 : Le cas Nike

- **Problème** : Prédire le nombre de lacets nécessaires pour le mois.
- **Fuite** : Inclure "la quantité de cuir *utilisée* ce mois-ci" comme caractéristique. Le cuir est utilisé *en même temps* que les lacets. Cette donnée ne sera pas connue au début du mois.
- **Solution potentielle** : Utiliser "la quantité de cuir *commandée* le mois précédent".
- **Réflexion** : Cela fonctionne *uniquement* si le cuir est commandé *avant* les lacets. La clé est l'ordre chronologique des événements.

Exemple 2 : Le cas du chirurgien

- **Problème** : Prédire si un patient aura une infection.
- **Fuite** : Créer une caractéristique "taux d'infection moyen du chirurgien" en utilisant *toutes* les opérations de ce chirurgien, y compris celle du patient que l'on essaie de prédire. Le résultat de l'opération (cible) est utilisé pour créer une caractéristique, c'est une fuite de cible.

Exemple 3 : Le cas de la Cryptomonnaie

- **Problème** : Prédire le prix d'une crypto-monnaie.
- **Pas de fuite** : Utiliser "Prix actuel", "Changement dans la dernière heure", "Volume des dernières 24h" n'est **pas** une fuite. Toutes ces informations sont connues au moment de la prédiction.
- **Attention** : Le modèle peut quand même être inutile. S'il prédit que le prix de demain sera 100\$ (et que le prix d'aujourd'hui est 100\$), il peut avoir un MAE faible ($<1\$$) mais être incapable de dire si le prix va *monter* ou *baissier*, ce qui est la seule information utile pour investir.

2.6.2 Contamination Train-Test

La contamination est une forme plus subtile de fuite où les données de validation influencent le prétraitement.

Exemple Si vous calculez la moyenne d'une colonne pour l'imputation *avant* de diviser vos données en `train_test_split`, la moyenne est calculée en utilisant *tout* le jeu de données. Votre ensemble d'entraînement contient donc une information (la moyenne) qui provient de votre ensemble de validation.

Méthodologie : La Solution des Pipelines La meilleure façon d'éviter la contamination est d'effectuer *tout* le prétraitement (imputation, encodage) **à l'intérieur** d'un pipeline. Lorsque vous appelez `my_pipeline.fit(X_train, y_train)`, le pipeline "apprend" les statistiques (comme la moyenne d'imputation) *uniquement* sur `X_train`. Cela garantit une séparation stricte et un modèle évaluable.

3 Explicabilité des Modèles (Machine Learning Explainability)

Ce chapitre explore les techniques permettant d'extraire des informations de modèles de machine learning complexes. Comprendre **pourquoi** un modèle prend une décision est souvent aussi important que la prédiction elle-même.

3.1 Introduction

L'explicabilité (ou interprétabilité) des modèles nous permet de répondre à des questions cruciales :

- Quelles caractéristiques le modèle juge-t-il les plus importantes ?
- Pour une prédiction unique, comment chaque caractéristique a-t-elle influencé ce résultat spécifique ?
- Quel est l'effet global de chaque caractéristique sur l'ensemble des prédictions ?

Ces informations sont précieuses pour plusieurs raisons : le **débogage** du modèle, l'orientation du **feature engineering**, la planification de **futures collectes de données** , l'aide à la **décision humaine** et le renforcement de la **confiance** dans le modèle.

3.2 "Importance par Permutation"

L'importance par permutation est une technique simple et efficace pour évaluer l'importance globale d'une caractéristique.

Concept Elle fonctionne en "brouillant" aléatoirement les valeurs d'une seule colonne dans l'ensemble de validation et en mesurant la dégradation de la performance du modèle.

1. Un modèle est entraîné (par exemple, un RandomForest).
2. L'score de performance (ex : "accuracy") est calculé sur les données de validation.
3. Les valeurs d'une caractéristique (ex : "Goal Score") sont permutées aléatoirement.
4. Le score est recalculé sur ces données "brouillées".
5. L'importance est la **différence** entre le score original et le score sur données brouillées.

Implémentation et Interprétation Nous utilisons la bibliothèque `eli5` pour calculer l'importance par permutation.

```
1 import eli5
2 from eli5.sklearn import PermutationImportance
3 from sklearn.ensemble import RandomForestClassifier
4
5 # ... (Entraînement du my_model sur train_X, train_y) ...
6
7 # Calcul de l'importance sur les données de validation
8 perm = PermutationImportance(my_model, random_state=1).fit(val_X, val_y)
```

```
9
10 # Affichage des poids
11 eli5.show_weights(perm, feature_names = val_X.columns.tolist())
```

Listing 3.1 – "Calcul de l'importance par permutation avec eli5"

L'interprétation du résultat se fait comme suit :

- **Valeurs hautes (ex : 0.1750)** : Caractéristiques les plus importantes. Le score a chuté de 17.5% lorsque cette colonne a été permutée.
- **Le '±' (ex : 0.0848)** : Indique la variabilité de la mesure sur plusieurs permutations.
- **Valeurs négatives** : Apparaissent lorsque, par pure chance, les prédictions sur les données brouillées étaient meilleures. Cela arrive typiquement pour des caractéristiques non importantes (importance proche de 0) et sur de petits jeux de données.

3.3 Graphiques de Dépendance Partielle (Partial Plots)

Si l'importance par permutation nous dit *quelles* caractéristiques sont importantes, les graphiques de dépendance partielle (PDP) montrent *comment* elles affectent les prédictions.

Concept Un PDP montre la relation marginale entre une ou deux caractéristiques et la prédiction du modèle, en "neutralisant" l'effet de toutes les autres caractéristiques. Il répond à la question : "En contrôlant tous les autres facteurs, quel est l'impact de la latitude sur le prix d'une maison ?"

3.3.1 PDP à une dimension (1D)

Nous utilisons `sklearn.inspection.PartialDependenceDisplay` pour visualiser l'effet d'une seule variable.

```
1 from matplotlib import pyplot as plt
2 from sklearn.inspection import PartialDependenceDisplay
3
4 # ... (Entraînement d'un tree_model) ...
5
6 feature_to_plot = 'Distance Covered (Kms)'
7 disp = PartialDependenceDisplay.from_estimator(tree_model, val_X, [feature_to_plot])
8 plt.show()
```

Listing 3.2 – "Creation d'un PDP 1D"

Méthodologie : Comparaison de modèles Les PDP sont excellents pour comparer la complexité des modèles. Un PDP basé sur un `DecisionTreeClassifier` simple montrera des paliers et des sauts clairs, reflétant exactement la structure de l'arbre. En revanche, un `RandomForestRegressor` montrera une courbe plus lisse et plus complexe, capturant des relations plus nuancées.

3.3.2 PDP à deux dimensions (2D)

Les PDP 2D sont utiles pour visualiser les **interactions** entre deux caractéristiques.


```
1 fig, ax = plt.subplots(figsize=(8, 6))
2 f_names = [('Goal Scored', 'Distance Covered (Kms)')]
3
4 # Le code est similaire, mais on passe un tuple de caracteristiques
5 disp = PartialDependenceDisplay.from_estimator(tree_model, val_X, f_names, ax=ax)
6 plt.show()
```

Listing 3.3 – "Creation d'un PDP 2D pour les interactions"

Ce type de graphique montre la prédiction (souvent par des courbes de niveau) pour chaque combinaison des deux caractéristiques. Par exemple, on peut y voir que la prédiction la plus élevée est atteinte lorsqu'une équipe marque au moins 1 but *et* court environ 100km.

3.3.3 Interprétation

- **PDP vs Importance** : Un PDP avec une forte pente ne garantit **pas** une importance par permutation élevée. La caractéristique pourrait avoir un gros effet, mais ne jamais varier dans les données réelles (ex : avoir la même valeur 99% du temps). Dans ce cas, la permuter n'aurait que peu d'impact.
- **PDP plat vs Importance élevée** : Inversement, un PDP peut être plat alors que la caractéristique est très importante. C'est le signe d'une **interaction** complexe. Par exemple, si 'y = X1 * X2', où 'X2' est soit 1 ou -1. L'effet de 'X1' est soit positif, soit négatif. En moyenne (ce que montre le PDP), l'effet s'annule et le graphique est plat. Pourtant, 'X1' est cruciale pour la prédiction, ce que l'importance par permutation détectera.

3.4 Valeurs SHAP (SHAP Values)

Les valeurs SHAP (SHapley Additive exPlanations) vont encore plus loin : elles décomposent une **prédiction individuelle** pour montrer la contribution de chaque caractéristique à ce résultat.

Concept Elles répondent à la question : "Pourquoi *ce patient* a-t-il été classé à haut risque ?". Les valeurs SHAP garantissent que la somme des contributions de chaque caractéristique est égale à la différence entre la prédiction du modèle pour cette observation et la prédiction "de base" (la moyenne des prédictions sur tout le dataset).

$$\sum(\text{Valeurs SHAP}) = \text{Prédiction} - \text{Prédiction de base}$$

3.4.1 SHAP Force Plot

Le "force plot" est la visualisation standard pour une seule prédiction. Il montre comment les caractéristiques "poussent" la prédiction loin de la valeur de base, soit vers un score plus élevé (en bleu), soit vers un score plus bas (en rouge).

```
1 import shap # Package pour les valeurs SHAP
2
3 # ... (Entrainement de my_model) ...
4
5 # Choisir une observation spécifique
6 row_to_show = 5
7 data_for_prediction = val_X.iloc[row_to_show]
8
9 # 1. Creer un "explainer" adapte au modele (ex: TreeExplainer pour les Forets)
```

```
10 explainer = shap.TreeExplainer(my_model)
11
12 # 2. Calculer les valeurs SHAP pour cette observation
13 # shap_values[0] = contributions pour la classe 0 (ex: "Non")
14 # shap_values[1] = contributions pour la classe 1 (ex: "Oui")
15 shap_values = explainer.shap_values(data_for_prediction)
16
17 # 3. Initialiser JS et afficher le graphique de force
18 shap.initjs()
19 shap.force_plot(explainer.expected_value[1], shap_values[1], data_for_prediction)
```

Listing 3.4 – "Calcul et affichage d'un SHAP force plot"

3.4.2 Interprétation

Les différentes techniques d'explicabilité s'enchaînent logiquement pour fournir un rapport complet à des experts métiers (ex : des médecins).

1. **Construire le modèle** pour prédire la réadmission.
2. **Utiliser l'Importance par Permutation** pour identifier les caractéristiques globalement importantes (ex : `number_inpatient`).
3. **Utiliser les PDP** pour montrer aux médecins *comment* cette caractéristique influence le modèle (ex : le risque augmente avec le nombre de procédures).
4. **Valider le modèle** en comparant les PDP aux données brutes. Si le PDP de `time_in_hospital` est plat, on peut vérifier le taux de réadmission moyen par jour d'hospitalisation pour confirmer que ce n'est pas une erreur du modèle.
5. **Créer une fonction** `patient_risk_factors(model, patient_data)` qui utilise un `shap.force_plot` pour expliquer le risque de *chaque patient individuellement*, permettant aux médecins de cibler les facteurs de risque.

3.5 Usages Avancés des Valeurs SHAP

En agrégeant les valeurs SHAP de nombreuses observations, on obtient des graphiques globaux plus riches que l'importance par permutation ou les PDP.

3.5.1 SHAP Summary Plot

Le "summary plot" remplace l'importance par permutation. Il montre non seulement l'importance, mais aussi la **direction** et la **magnitude** de l'effet.

```
1 # Calculer les valeurs SHAP pour tout l'ensemble de validation
2 shap_values = explainer.shap_values(val_X)
3
4 # Afficher le summary plot pour la classe positive (index [1])
5 shap.summary_plot(shap_values[1], val_X)
```

Listing 3.5 – "Creation d'un SHAP summary plot"

Interprétation Ce graphique est un nuage de points où :

- **Axe Y** : Les caractéristiques, triées par importance.
- **Axe X** : La valeur SHAP (l'impact sur la prédiction).
- **Couleur** : La valeur de la caractéristique (ex : Rouge = élevée, Bleu = faible).

On peut y lire que des valeurs élevées (points rouges) de "Goal Scored" ont un impact positif (points à droite de 0), et des valeurs faibles (points bleus) ont un impact négatif.

3.5.2 SHAP Dependence Plot

Le "dependence plot" est une alternative au PDP. Il montre l'impact d'une caractéristique (Axe Y = Valeur SHAP) en fonction de sa valeur (Axe X = Valeur Caractéristique). Son atout majeur est d'utiliser la couleur pour révéler une **interaction** avec une seconde caractéristique.

```
1 # 'Ball Possession %' = caractéristique principale
2 # 'Goal Scored' = caractéristique d'interaction pour la couleur
3 shap.dependence_plot('Ball Possession %', shap_values[1], X,
4                       interaction_index="Goal Scored")
```

Listing 3.6 – "Creation d'un SHAP dependence plot"

Méthodologie d'interprétation

- **Détecter les interactions** : Si, pour une même valeur sur l'axe X (ex : 50% de possession), on observe une grande dispersion verticale des points (certains ont un SHAP positif, d'autres négatif), cela signale une interaction. La couleur nous dit *quelle* autre variable est responsable de cette différence.
- **Interpréter les interactions** : On peut voir que "Haute possession" (ex : 70%) a généralement un impact positif (SHAP > 0), mais si ces points sont bleus (indiquant "Goal Scored" = 0), l'impact devient négatif. Le modèle a appris que dominer sans marquer est pénalisé.
- **Points "brouillés"** : Si un "summary plot" montre des points bleus et rouges mélangés des deux côtés de l'axe zéro (ex : pour `num_lab_procedures`), c'est le signe d'un fort effet d'interaction, qu'un "dependence plot" peut aider à clarifier.